## 1. Write a shell script to write the factorial of number

**Create a new shell script file**

nano factorial.sh

**Write the script:**

```
#!/bin/bash
read -p "Enter a number: " n
f=1
for ((i=1;i<=n;i++)); do f=$((f*i));
done
echo "Factorial of $n is $f"
```

**Save and exit Nano**

**1. Press Ctrl + O → Enter (to save)**

**2. Press Ctrl + X (to exit Nano)**
**---**

**Step 5: Give execute permission**

chmod +x factorial.sh

---

**Step 6: Run the script**

./factorial.sh

## 2. Write the shell script to check whether the number is perfect or not

**create a file name**:

nano perfect.sh

**write the script inside it:**

```
echo "Enter a number: "
read n
sum=0
for i in $(seq 1 $((n-1)))
do
  rem=$(( n % i ))
  if [ $rem -eq 0 ]
  then
    sum=$(( sum + i ))
```

```
  fi
done
if [ $sum -eq $n ]
then
  echo "$n is perfect"
else
  echo "$n is not perfect"
fi
```

**then press ctrl+o,enter,ctrl+save then ctrl+x  to exit :**

chmod +x perfect.sh

**then , put**

./perfect.sh

**output:**

```
mcs14@pc11:~$ ./perfect.sh
Enter a number:
6
6 is perfect
mcs14@pc11:~$ ./perfect.sh
Enter a number:
99
99 is not perfect
mcs14@pc11:~$
```

## 3. Write the shell script to find a maximum of three numbers

**create a file name:**

nano max3.sh

**write the script inside it:**

```
#!/bin/bash
read -p "Enter first number: " a
read -p "Enter second number: " b
read -p "Enter third number: " c
max=$a
if [ $b -gt $max ]; then
  max=$b
fi
if [ $c -gt $max ]; then
  max=$c
fi
echo "The maximum number is: $max"
```

**then press ctrl+o,enter,ctrl+save then ctrl+x  to exit :**

```
chmod +x max3.sh
```

**then , put**

```
./max3.sh
```

**output:**

```
mcs14@pc11:~$ ./max3.sh
Enter first number: 6
Enter second number: 7
Enter third number: 9
The maximum number is: 9
```

## 4. Create a docker file to build a given nodejs project

```
mkdir my-node-app
cd my-node-app
npm init -y
docker buildx build -t my-node-app .
touch Dockerfile
```

**in folder create the app.js file :**

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.end('Hello from Node.js Docker App!');
});

server.listen(3000, () => console.log('Server running on port 3000'));
```

**then in the Dockerfile put,**

```
FROM node:22
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "app.js"]
```

**Then in cmd,**

```
docker build -t my-node-app .
```

```
docker run -p 3000:3000 my-node-app
```

## 5. Create a docker file to build the nodejs container

**In cmd,**

```
docker --version
```

**Create folder, in cmd**

```
mkdir my-node-app
cd my-node-app

npm init -y
```

**Then open folder in vs code and create app.js file**

```
const http = require('http');

const hostname = '0.0.0.0';
const port = 3000;

const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Hello from Node.js Docker container!\n');
});

server.listen(port, hostname, () => {
  console.log(`Server running at http://${hostname}:${port}/`);
})
```

**Then create Dockerfile**

```
FROM node:22
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "app.js"]
```

**Then in cmd,**

```
docker build -t my-node-app .

docker run -p 3000:3000 my-node-app
```

docker ps

## 6. create a shell script to count the number of words

**In cmd put,**

nano count_words.sh

**Now write in the file**

```
#!/bin/bash

read -p "Enter a sentence: " sentence

word_count=$(echo "$sentence" | wc -w)

echo "Number of words: $word_count"
```

**Save and Exit**

**Press Ctrl + O → Enter → Ctrl + X**

**Now in cmd,**

chmod +x count_words.sh

./count_words.sh

**output:**

Enter a sentence: DevOps is an automation tool
Number of words: 5

## 7. build and run docker container to ensure that nodejs app is running and accessible from browser

**# check docker installed**

docker --version

**# check docker service running**

sudo systemctl status docker --no-pager

```
sudo systemctl enable --now docker
```

### In cmd,

```
mkdir my-node-app
cd my-node-app
npm init -y
```

### In folder create the app.js:

```
const http = require('http');
const hostname = '0.0.0.0';
const port = 3000;
const server = http.createServer((req, res) => {
  res.writeHead(200, {'Content-Type': 'text/plain'});
  res.end('Hello from Node.js Docker container!\n');
});
server.listen(port, hostname, () => {
  console.log(`Server running at http://${hostname}:${port}/`);
});
```

### In folder create Dockerfile

```
FROM node:22
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "app.js"]
```

### In cmd,

```
docker build -t my-node-app .

docker run -d --name my-node -p 3000:3000 my-node-app

docker ps

docker stop my-node
docker rm my-node
```

## 8. create docker compose file  to contain mysql server

**In cmd,**

```
mkdir mysql-docker
cd mysql-docker
```

**Create a new file named docker-compose.yml**

```
version: "3.8"

services:
  mysql-server:
    image: mysql:8.0
    container_name: my-mysql
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: root   # root password
      MYSQL_DATABASE: testdb      # optional default database
    ports:
      - "3306:3306"               # host:container port
    volumes:
      - mysql-data:/var/lib/mysql

volumes:
  mysql-data:
```

**In cmd,**

```
docker-compose up -d

docker ps

docker exec -it my-mysql mysql -u root -p
```

**enter password: root**

**Test database:**

```
SHOW DATABASES;
USE testdb;
SHOW TABLES;


docker-compose logs -f

docker-compose down
```

## 9. run any 4 github commands and print logs from it

```
sudo apt update

sudo apt install git -y

git --version
```

**Initialize a Git repository**

```
git init
```

**Check status**

```
git status
```

**Add a file Create a file (example):**

```
echo "Hello Git" > file1.txt

git add file1.txt

git status
```

**Commit the file**

```
git commit -m "Initial commit with file1.txt"
```

**Check log**

```
git log
```

## 10. resolve conflict while committing current branch for the given project

```
git status                  # see conflicted files
# open conflicted files and manually resolve them
git add <file>              # stage resolved files
git status                  # confirm all conflicts resolved
git commit -m "Resolved conflict"  # commit resolution
git log --oneline           # verify commit
git push origin <branch>    # push to remote
```

## 11.create github actions that runs every time the code is pushed to main branch and run nodejs test

```
node -v
npm -v

mkdir my-node-app
cd my-node-app

npm init -y

touch app.js
touch test.js
```

**Open vs code same folder and open app.js**

```
console.log("Hello, Node.js project is running!");
```

**Open test.js and add a simple test:**

```
console.log("Test is running successfully!");
```

**Open package.json and add under "scripts":**

```
"scripts": {
  "test": "node test.js"
}
```

**Run in terminal:**

```
node app.js
npm test
```

```
node -v
npm -v

npm init -y

touch app.js
touch test.js
```

**Open the vs code and open app.js**

```
const http = require('http');
const hostname = '0.0.0.0';
const port = 3000;

const server = http.createServer((req, res) => {
  res.writeHead(200, {'Content-Type': 'text/plain'});
  res.end('Hello from Node.js app deployed via GitHub Actions!\n');
});

server.listen(port, hostname, () => {
  console.log(`Server running at http://${hostname}:${port}/`);
});
```

**Then open [test.js]**

```
console.log("Test is running successfully!");
```

**Open package.json and add under "scripts":**

```
"scripts": {
  "test": "node test.js"
}
```

## 12. create the github actions that builds nodejs project and deploy it on aws

**In terminal run,**

```
node app.js
npm test
```

**In cmd,**

```
git init
git add .
git commit -m "Initial commit"

git remote add origin https://github.com/YOUR_USERNAME/my-node-app.git
git branch -M main
git push -u origin main
```

**Prepare AWS EC2 instance**

```
sudo apt update
sudo apt install -y nodejs npm git
```

```
sudo npm install -g pm2
```

**Create GitHub Actions workflow**

```
mkdir -p .github/workflows
```

**Create .github/workflows/deploy.yml:**

```
name: Build and Deploy Node.js to AWS

on:
  push:
    branches:
      - main

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v3

      - name: Setup Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '22'

      - name: Install dependencies
        run: npm install

      - name: Run tests
        run: npm test

      - name: Deploy to AWS EC2
        uses: appleboy/ssh-action@v0.1.7
        with:
          host: ${{ secrets.AWS_HOST }}
          username: ${{ secrets.AWS_USER }}
          key: ${{ secrets.AWS_KEY }}
          port: 22
          script: |
            mkdir -p ${{ secrets.AWS_PATH }}
            cd ${{ secrets.AWS_PATH }}
            git pull origin main || git clone https://github.com/${{ github.repository }} .
            npm install
```

```
pm2 stop all || true
pm2 start app.js --name my-node-app
```

**Commit and push workflow**

```
git add .github/workflows/deploy.yml
git commit -m "Add GitHub Actions workflow for build and deploy to AWS"
git push origin main

ssh -i /path/to/key.pem ubuntu@EC2_PUBLIC_IP
pm2 list
```

**Open browser**

http://EC2_PUBLIC_IP:3000

## 13. create github workflow that that runs nodejs lint errors and store them in a file

**In cmd**

```
node -v
npm -v

mkdir my-node-app
cd my-node-app
npm init -y

touch app.js
```

**Open folder and put in app.js**

```
const message = "Hello, Node.js Lint Workflow!";
console.log(message);
```

**In cmd again**

```
npm install eslint --save-dev

npx eslint --init
```

**Choose options:**

- **How would you like to use ESLint? → To check syntax and find problems**

- **Type of modules → CommonJS**
- **Framework → None**
- **TypeScript → No**
- **Where will your code run? → Node**
- **Format → JSON**

**This creates .eslintrc.json.**

**Open cmd**

npx eslint app.js

**Open package.json and add under "scripts":**

```
"scripts": {
  "lint": "eslint . --format json > lint-report.json || true"
}
```

**In terminal**

npm run lint

**Create the workflow folder in cmd**
mkdir -p .github/workflows

**Create .github/workflows/lint.yml:**

```
name: Node.js Lint Check

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main

jobs:
  lint:
    runs-on: ubuntu-latest
```

```
  steps:
    - name: Checkout repository
      uses: actions/checkout@v3

    - name: Setup Node.js
      uses: actions/setup-node@v3
      with:
        node-version: '22'

    - name: Install dependencies
      run: npm install

    - name: Run ESLint
      run: npm run lint

    - name: Upload Lint Report
      uses: actions/upload-artifact@v3
      with:
        name: lint-report
        path: lint-report.json
```

**Open cmd,**

```
git add .github/workflows/lint.yml package.json .eslintrc.json app.js
git commit -m "Add ESLint workflow and lint script"
git push origin main
```

## 14. create a jenkins pipeline to check out the code from github and start node app

**Install Jenkins**

**On Ubuntu:**

```
sudo apt update
sudo apt install openjdk-11-jdk -y
wget -q -O - https://pkg.jenkins.io/debian/jenkins.io.key | sudo apt-key add -
sudo sh -c 'echo deb http://pkg.jenkins.io/debian-stable binary/ >
/etc/apt/sources.list.d/jenkins.list'
sudo apt update
sudo apt install jenkins -y
sudo systemctl start jenkins
sudo systemctl enable jenkins
```

- Open browser: http://localhost:8080
- Unlock Jenkins using:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

- Install suggested plugins

---

**Step 2: Install Required Plugins in Jenkins**

Go to **Manage Jenkins → Manage Plugins → Available**:

1. **NodeJS Plugin** → for Node.js management
2. **Git Plugin** → for GitHub checkout
3. **Pipeline Plugin** → for pipeline jobs

Install them and restart Jenkins if required.

---

**Step 3: Configure Node.js in Jenkins**

1. Go to **Manage Jenkins → Global Tool Configuration → NodeJS**
2. Add NodeJS installation:
    - Name: NodeJS
    - Check **Install automatically** and choose latest Node version

---

**Step 4: Create Node.js Project (if not created yet)**

```
mkdir my-node-app
cd my-node-app
npm init -y
touch app.js
```

Add sample code in app.js:

```
const http = require('http');
const hostname = '0.0.0.0';
const port = 3000;

const server = http.createServer((req, res) => {
  res.writeHead(200, {'Content-Type': 'text/plain'});
  res.end('Hello from Node.js app!\n');
});

server.listen(port, hostname, () => {
  console.log(`Server running at http://${hostname}:${port}/`);
});
```

Test locally:

```
node app.js
```

---

**Step 5: Push Project to GitHub**

```
git init
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/my-node-app.git
git push -u origin main
```

---

**Step 6: Create Jenkinsfile**

In project root, create Jenkinsfile:

```
pipeline {
    agent any

    environment {
        NODEJS_HOME = tool name: 'NodeJS', type: 'NodeJS'
        PATH = "${NODEJS_HOME}/bin:${env.PATH}"
    }

    stages {
        stage('Checkout') {
            steps {
```

```
            git branch: 'main', url: 'https://github.com/YOUR_USERNAME/my-node-app.git'
        }
    }

    stage('Install Dependencies') {
        steps {
            sh 'npm install'
        }
    }

    stage('Start Node App') {
        steps {
            // Kill previous node processes if running
            sh 'pkill node || true'
            // Start Node app in background
            sh 'nohup node app.js > app.log 2>&1 &'
        }
    }
  }

  post {
    always {
        echo 'Pipeline finished.'
    }
  }
}
```

- Replace YOUR_USERNAME and repository URL accordingly

---

**Step 7: Configure Jenkins Pipeline Job**

1. Go to **Jenkins Dashboard → New Item → Pipeline**
2. Name: node-app-pipeline → Select **Pipeline** → OK
3. In **Pipeline → Definition → Pipeline script from SCM**:
    - SCM: **Git**
    - Repository URL: https://github.com/YOUR_USERNAME/my-node-app.git
    - Branch: main

- Script Path: Jenkinsfile

---

**Step 8: Run the Pipeline**

1. Click **Build Now** in Jenkins
2. Check **Console Output**:
    - Code will be checked out
    - Dependencies installed
    - Node.js app started in background

---

**Step 9: Verify Node.js App**

1. SSH into Jenkins server if remote:

```
ps -ef | grep node
tail -f app.log
```

2. Open browser:

http://JENKINS_SERVER_IP:3000

You should see:

Hello from Node.js app!

## 15. create a github repository and push files to it

A **GitHub account** → https://github.com
git --version

sudo apt update && sudo apt install git -y  # For Ubuntu/Debian

**Step 1: Create a new GitHub repository**

1. Go to **https://github.com/**
2. Click **New** (or the "+" icon → *New repository*).
3. Fill the form:
   - **Repository name** → e.g. devops-practical
   - Description → *(optional)*
   - Choose **Public** or **Private**
   - **DO NOT** initialize with a README (we'll add it manually)
4. Click **Create repository**

### Step 2: Set up Git locally (Open your terminal)

```
git config --global user.name "Your Name"
git config --global user.email "your_email@example.com"
```

### Step 3: Create a local project folder

```
mkdir devops-practical
cd devops-practical
```

### Step4: Add a file

```
echo "# DevOps Practical Repository" > README.md
echo "This repo is for my DevOps practicals." >> README.md
```

### Step 5: Initialize Git & make your first commit

```
git init
git add .
git commit -m "Initial commit - added README and main.py"
```

### Step 6: Connect your local repo to GitHub

**From your GitHub repo page, copy the HTTPS URL — looks like this:**

https://github.com/<your-username>/devops-practical.git

**Now link it:**

git remote add origin https://github.com/<your-username>/devops-practical.git

### Step 7: Push your files to GitHub

```
git branch -M main
git push -u origin main
```

**If asked for username/password:**

- Username → your GitHub username

## 16. write a shell script to print below pattern

**0**
**0 1**
**0 1 2**
**0 1 2 3**

## Create a file in terminal

```
mkdir -p ~/devops-practicals
cd ~/devops-practicals
nano pattern.sh
```

pattern.sh

```
#!/bin/bash
for i in {0..3}
do
  for j in $(seq 0 $i)
  do
    echo -n "$j "
  done
  echo
done
```

**Save and exit (Ctrl+O, Enter, Ctrl+X in nano).**

### Again in terminal:

```
chmod +x pattern.sh

./pattern.sh
```

## 17. Clone an existing public repository and pull files from it

git --version

https://github.com/octocat/Spoon-Knife.git

**Open your terminal and run:**

```
git clone https://github.com/octocat/Spoon-Knife.git

cd Spoon-Knife

ls

git pull origin main

git status
```

**On branch main**
**Your branch is up to date with 'origin/main'.**

## 18. write a shell script to compile all c files and delete those files having errors

```
mkdir -p ~/devops-practicals
cd ~/devops-practicals
nano compile_c.sh
```

**Code:**

```
#!/bin/bash

shopt -s nullglob
c_success=0
c_failed=0

echo "Starting compilation of .c files in $(pwd)"

for file in ./*.c; do
  echo "----------------------------"
  echo "Compiling: $file"

  errlog=$(mktemp /tmp/compile_err.XXXXXX)
  gcc -Wall "$file" -o "${file%.c}.out" 2> "$errlog"
```

```
  rc=$?

  if [ $rc -ne 0 ]; then
    echo " Compilation FAILED for: $file (exit code $rc)"
    echo "Deleting source file: $file"
    rm -f "$file"
    ((c_failed++))
    echo "Error summary:"
    sed -n '1,20p' "$errlog"
  else
    echo " Compilation succeeded for: $file -> ${file%.c}.out"
    ((c_success++))
  fi

  rm -f "$errlog"
done

if [ $c_success -eq 0 ] && [ $c_failed -eq 0 ]; then
  echo "No .c files found in the current directory."
else
  echo "----------------------------"
  echo "Summary: $c_success succeeded, $c_failed failed and deleted."
fi
```

**Then save and press Ctrl + O → Enter → Ctrl + X**

**Then in terminal**

```
chmod +x compile_c.sh

./compile_c.sh
```

## 19. Illustrate the use of any 3 git commands add, commit, push, pull

### Create a new folder

```
mkdir git-demo
cd git-demo
git init

echo "Hello Git!" > hello.txt

git add hello.txt
```

```
git commit -m "Added hello.txt file"

git remote add origin https://github.com/your-username/git-demo.git
git push -u origin main

git pull origin main
```

**20. write a shell script to print below pattern**
**1**
**1 2 1**
**1 3 3 1**

**Create a file:**

```
nano pattern.sh

#!/bin/bash
# Script to print Pascal's Triangle pattern (first 3 rows)

rows=3
for (( i=0; i<rows; i++ ))
do
  num=1
  for (( j=0; j<=i; j++ ))
  do
    echo -n "$num "
    num=$(( num * (i - j) / (j + 1) ))
  done
  echo
done
```

**Save and press ctrl+o enter ctrl+x**

**Again in terminal**

```
chmod +x pattern.sh

./pattern.sh
```

## 21. write a docker compose file for mysql user creation

### Create a new project folder in terminal

```
mkdir mysql-compose-demo
```

cd mysql-compose-demo

**Create a docker-compose.yml file**

nano docker-compose.yml

**Paste this code**

```
version: '3.8'

services:
  mysql:
    image: mysql:8.0
    container_name: mysql_container
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: root123          # Root password
      MYSQL_DATABASE: testdb                # Auto-create this database
      MYSQL_USER: devuser                   # Auto-create this user
      MYSQL_PASSWORD: devpass123            # Password for devuser
    ports:
      - "3306:3306"                         # Expose MySQL port
    volumes:
      - mysql_data:/var/lib/mysql           # Persist data even after container restart
volumes:
  mysql_data:
```

**Step 3: Run the container**

docker-compose up -d
docker ps
docker exec -it mysql_container mysql -u devuser -p  #(**ask for password of database)**
SHOW DATABASES;

## 22. create a github release for any repository and use tag 1.0.0

1. **Go to your repository** on GitHub.
   Example:
   👉 https://github.com/your-username/your-repo

2. Click on the **"Releases"** tab (on the right side under "About").

3. Click **"Create a new release"**.

4. In the **"Tag version"** field, type:

   1.0.0 (If this tag doesn't exist yet, GitHub will create it.)

5. Choose the branch you want to release (usually main or master).

6. Add a **Release title** and **Description**, e.g.

- **Title:** Version 1.0.0
- **Description:** Initial stable release

7. Click **"Publish release"**.

## 23. Start a docker container of any image and expose port 9000 from it

```
docker pull nginx
docker run -d -p 9000:80 --name mynginx nginx
docker ps
docker stop mynginx
docker rm mynginx
```

## 24. create a git branch from master branch

Go to your local repository folder:

```
cd path/to/your/repository
git branch
```

**You should see:**

* master

**Use the following command:**

```
git branch feature-branch
```

### Switch to the new branch

```
git checkout feature-branch

git branch
```

## 25. execute the below docker commands docker exec,docker rm, docker rmi

In terminal:

```
docker ps
docker exec -it mynginx bash
ls
exit
docker stop mynginx
docker rm mynginx
docker rm -f mynginx
docker rmi nginx
```

## 26. execute and store the output of the following commands to a file - ls, pwd,who,echo hello

nano save_output.sh

```
#!/bin/bash
# save_output.sh

# Store output of commands in output.txt
{
  echo "Output of ls:"
  ls
  echo

  echo "Output of pwd:"
  pwd
  echo

  echo "Output of who:"
  who
  echo

  echo "Output of echo hello:"
  echo hello
} > output.txt
```

**In terminal:**

```
chmod +x save_output.sh
./save_output.sh
cat output.txt
```

## 27. write a shell script to count the number words, lines, characters

**Open terminal and create file**

nano count_file_stats.sh

Code:

```
#!/bin/bash

echo "Enter the filename:"
read filename

if [ -f "$filename" ]; then
   lines=$(wc -l < "$filename")
   words=$(wc -w < "$filename")
   chars=$(wc -m < "$filename")

   echo "File: $filename"
   echo "Number of lines: $lines"
   echo "Number of words: $words"
   echo "Number of characters: $chars"
else
   echo " File not found!"
fi
```

**Save ctrl +o enter ctrl+x**

**Again in terminal**

chmod +x count_file_stats.sh

./count_file_stats.sh

## 28. write a shell to reverse a string

**Create a file in terminal**

nano reverse_string.sh

**Code:**

```
#!/bin/bash
# reverse_string.sh
# This script reverses a given string.
```

```
echo "Enter a string:"
read str

# Method 1: Using 'rev' command
rev_str=$(echo "$str" | rev)

echo "Reversed string: $rev_str"
```

**Save and ctrl+o enter ctrl+x**
**Again in terminal**

```
chmod +x reverse_string.sh
./reverse_string.sh
```

## 29. use the bellow commands chmod,  chown grep

**Create a file with the name sample.txt and put inside the file**

```
echo "Hello world
This is a test file
Linux commands are powerful
Hello again" > sample.txt
```

**Again in terminal**

```
cat sample.txt
chmod 744 sample.txt
ls -l sample.txt
sudo chown root sample.txt
ls -l sample.txt
grep "Hello" sample.txt
```

## 30. write the shell script to print element of array

```
nano print_array.sh

#!/bin/bash

fruits=("Apple" "Banana" "Cherry" "Mango" "Orange")

echo "All elements of the array:"
echo "${fruits[@]}"

echo "Printing each element using for loop:"
for fruit in "${fruits[@]}"
do
```

```
  echo "$fruit"
done
```

**Save and exit (Ctrl + O, Enter, Ctrl + X).**

**Again in terminal**

```
chmod +x print_array.sh
./print_array.sh
```

## 31. write a shell script to change case of string

**Create a file**

Nano case_string.sh

```
#!/bin/bash

echo "Enter a string:"
read str

echo "Choose an option:"
echo "1. Convert to UPPERCASE"
echo "2. Convert to lowercase"
read choice

if [ "$choice" -eq 1 ]; then
  echo "Result: ${str^^}"
elif [ "$choice" -eq 2 ]; then
  echo "Result: ${str,,}"
else
  echo "Invalid choice!"
fi
```

**Save and exit (Ctrl + O, Enter, Ctrl + X).**

**Again in terminal**

```
chmod +x case_string.sh
./case_string.sh
```